Google Summer of Code

Project Proposal

🥪BLT-Hackerhouse: Real time contributor visualization platform

Organisation - OWASP

Applicant - Rudra Pratap Singh

Project Size - Large

Lead Mentor - Donnie Brown



Project Description

The Problem

BLT maintainers today operate across at least four disconnected surfaces: GitHub activity, Slack coordination, BLT workflow signals, and leaderboard data. Understanding the current ecosystem state requires manually switching between these tools, and by the time a bottleneck is visible, it has often already slowed the pipeline. New contributors have no way to see at a glance where help is needed or where a contribution is stuck.

This Causes three concrete failures:

1. No single live view of contributor workflow state across all stages.
2. Bottlenecks are detected late because signals are fragmented across tools.
3. New contributors cannot orient themselves quickly, increasing onboarding friction.

The Solution

HackerHouse is a realtime operational dashboard that ingests events from GitHub, Slack, and BLT, normalizes them into a unified internal model, resolves contributor identity across platforms, and renders live contributor movement across a three floor spatial interface. A maintainer can understand the current ecosystem state within five seconds of opening the dashboard without switching to any other tool.

why Not existing tools?

Tool	Primary Mode	What It Does Well	What HackerHouse Adds
Grafana	Observability metrics	Infrastructure telemetry and historical trends	Contributor workflow state mapped to rooms for immediate triage
GitHub Insights	Repository analytics	Retrospective repo metrics	Cross platform, realtime contributor movement across GitHub + Slack + BLT
LinearB	Engineering analytics	PR cycle analytics over time	Spatial, stage level bottleneck visibility designed for "what needs action now"
Orbit	Community CRM	Member engagement tracking	Live workflow progression and maintainer facing intervention signals

What Makes This Different

Realtime operational view, not retrospective analytics **Spatial workflow mapping** that makes stage congestion instantly visible **Cross platform identity unification** to avoid fragmented contributor personas **Snapshot + WebSocket delivery** designed for low latency maintainer action

Current State and Testing

Before writing a line of implementation code, I validated which components of the existing stack can realistically serve as integration points and where the actual friction lies. The following analysis is based on hands-on testing in my local environment.

What Currently Works

- GitHub webhook delivery and HMAC-SHA256 signature verification against a local FastAPI endpoint tested with real payloads from a test repository.

- SQLite event persistence with idempotency key deduplication tested with synthetic duplicate deliveries.

- FastAPI WebSocket broadcast to multiple connected clients tested with three concurrent browser tabs receiving live updates.

What required investigation!

- Identity resolution across platforms:

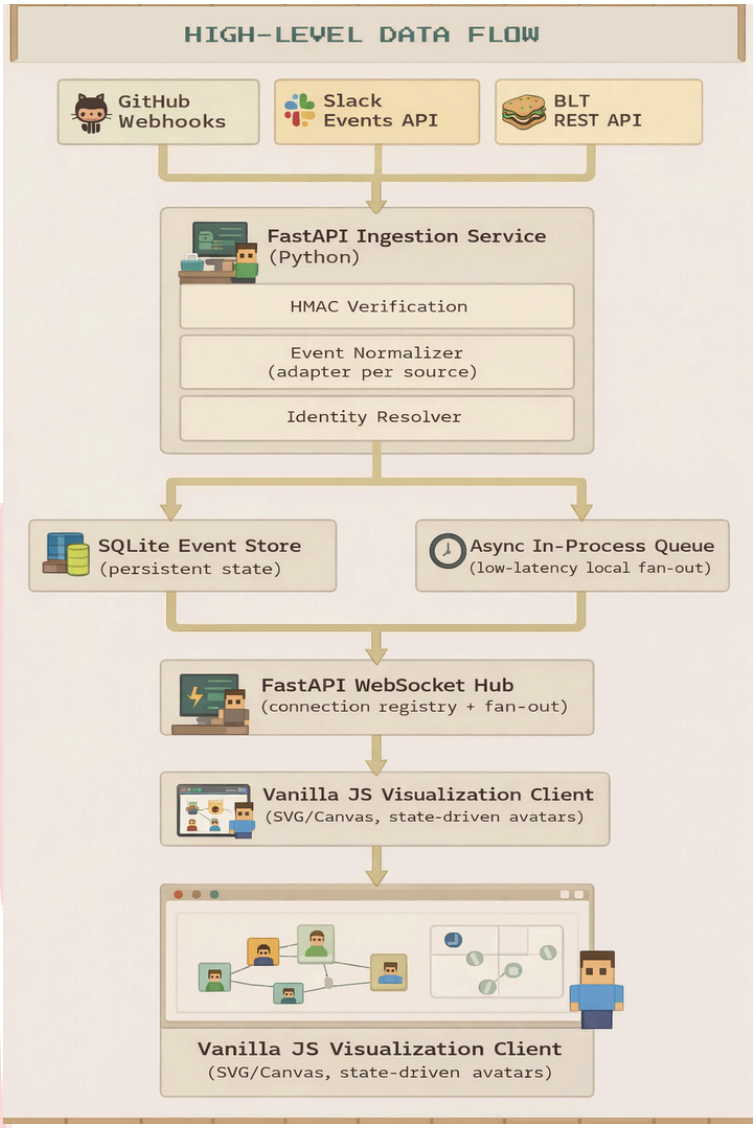
I manually mapped 30 real contributor pairs across GitHub and Slack usernames in the BLT ecosystem.
Findings: exact email match is available for approximately 40% of pairs. Of the remaining 60%, Levenshtein distance ≤ 2 on username handles approximately 55% of cases.
Token overlap on display names (Jaccard similarity ≥ 0.6) covers a further 20%. The remaining ~25% require maintainer confirmation this is expected and is handled explicitly by the unlinked avatar indicator.

- BLT signal integration: BLT exposes /api/issues/ and /api/hunts/ as REST endpoints.
- These are confirmed available and machine readable. Webhook push support for BLT
- events is not currently available, so BLT signals are ingested via a 60 second polling
- adapter. GitHub and Slack use push webhooks. BLT polling is explicitly scoped as
- additive its absence does not break the GitHub/Slack pipeline.

Non Goals

- Historical replay or time travel of past ecosystem events
- Avatar movement animations beyond state driven position updates
- Discord, Jira, or any integration beyond GitHub, Slack, and BLT
- Authentication or access control on the dashboard
- Advanced contributor achievement badges or gamification layers

High level data flow



Core Components

Component	Input	Core Mechanism	Output
Event Ingestion Service	GitHub webhooks, Slack events, BLT poll	HMAC-SHA256 verification → adapter dispatch → deduplication	Verified, deduplicated raw payload
Event Normalizer	Raw source payload	Source-specific adapter converts to Event schema	Validated Event object
Identity Mapper	Event actor fields	Email exact match → Levenshtein fuzzy match → Jaccard display name → confidence scoring	Unified or flagged contributor identity
SQLite State Layer	Normalized Event, identity	Persist events, update room state, maintain avatar positions	Persistent room + avatar state
Async Queue	Persisted events	In-process async fan-out to broadcast hub	Ordered event stream
WebSocket Broadcast Hub	Event stream, connection registry	Fan-out to all subscribed clients, snapshot on connect	Live state updates to all clients
Vanilla JS Renderer	WebSocket state updates	State-driven avatar position updates, room occupancy rendering	Live HackerHouse visual
Analytics Overlay	Room state, event history	Leaderboard projection, metrics aggregation	Maintainer visibility panel

Implementation Plan

1. Normalized Event Schema

All source specific payloads are converted into a single internal Event model before any further processing. Platform specific adapter logic is fully isolated the core pipeline never inspects raw payloads directly.

```
from pydantic import BaseModel
from datetime import datetime

class Event(BaseModel):
    id: str              #UUID generated at ingestion time #github | slack | blt
    source:str           #issue_opened | pr_opened | review_requested | ci_run|
    event_type:str
bug_bounty_submitted
    actor_id:str         #platform-native user ID #display name used for avatar
    actor_name:str       label #e.g. OWASP-BLT/BLT #UTC, parsed from source
    repository:str       payload #retained raw fields for routing and overlay
    timestamp:datetime   details
    metadata:dict
```

Field mapping by source

Source	actor_id	actor_name	repository
GitHub	sender.id	sender.login	repository.full_name
Slack	event.user	resolved via Users API	linked repo from message context
BLT	user.id	user.username	hunt.repo

Example normalized event:

```
{
  "id": "evt_a3f7c1", "source": "github",
  "event_type": "pr_opened", "actor_id":
  "gh_84291", "actor_name": "Rudra Pratap Singh",
  "repository": "OWASP-BLT/BLT", "timestamp":
  "2026-03-16T10:22:00Z", "metadata": {
    "pr_number": 412, "title": "Add webhook ingestion
    layer", "labels": ["backend"]
  }
}
```

2. Full SQLite Schema



3. Event Ingestion Service

Receives and verifies events from GitHub and Slack before any processing occurs. HMAC- SHA256 verification runs before payload parsing invalid signatures are rejected immediately with a 401.

```python
import hmac, hashlib, json
from fastapi import FastAPI, Request, HTTPException

app = FastAPI()

defverify_github_signature(payload: bytes, signature: str, secret: str) -> bool:
    expected = "sha256=" + hmac.new(
        secret.encode(), payload, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, signature)

@app.post("/webhook/github")
async def github_webhook(request: Request):
    payload = await request.body()
    sig = request.headers.get("X-Hub-Signature-256", "")
    if not verify_github_signature(payload, sig, GITHUB_SECRET):
        raise HTTPException(status_code=401, detail="Invalid signature")
    event = normalize_github(json.loads(payload))
    await queue.put(event)
    return {"status": "accepted"}
```

Deduplication uses an idempotency key derived from `source + event_type + actor_id + timestamp`. Duplicate deliveries from the same source are silently dropped before normalization.

4. Event Normalization

Each event source is handled through a dedicated adapter that translates raw payloads into the unified Event schema. This ensures that all downstream components operate on a consistent representation regardless of source specific structure.

For e.g., GitHub webhook actions are mapped to internal event types such as pr_opened, pr_merged, and issue_opened, while Slack and BLT events are similarly normalized into equivalent workflow signals. This abstraction layer isolates platform specific complexity and allows the core pipeline to remain source agnostic.

```
def map_github_event(action: str, payload: dict) -> str:
 if "pull_request" in payload and action == "opened":
 return "pr_opened"
 if "pull_request" in payload and action == "closed" and
payload["pull_request"].get("merged"):
 return "pr_merged"
 if "issue" in payload and action == "opened":
 return "issue_opened"
 return "unknown"


def normalize_github(payload: dict) -> Event:
 return Event(
 id=str(uuid4()),
 source="github",
 event_type=map_github_event(payload.get("action"), payload),
 actor_id=str(payload["sender"]["id"]),
 actor_name=payload["sender"]["login"],
 repository=payload["repository"]["full_name"],
 timestamp=datetime.utcnow(),
 )
```

5. Identity Resolution

This is the hardest problem in the pipeline. Without explicit identity linking, the same contributor appears as three separate avatars across GitHub, Slack, and BLT, fragmenting the house view and corrupting leaderboard data.

Resolution pipeline (executed in order):

Implementation uses rapidfuzz for steps 2 and 3:

```
from rapidfuzz.distance import Levenshtein
from rapidfuzz.fuzz import token_set_ratio

def resolve_identity(actor_id: str, actor_name: str, source: str, db):
    # 1. Exact match (highest confidence)
    row = db.find_by_platform_id(actor_id, source)
    if row:
        return row.id, 1.0

    candidates = db.get_all_identities()

    # 2. Username similarity (Levenshtein ≤ 2)
    for c in candidates:
        if any(Levenshtein.distance(actor_name.lower(), u.lower()) <= 2 for u in c.usernames):
            return c.id, 0.75

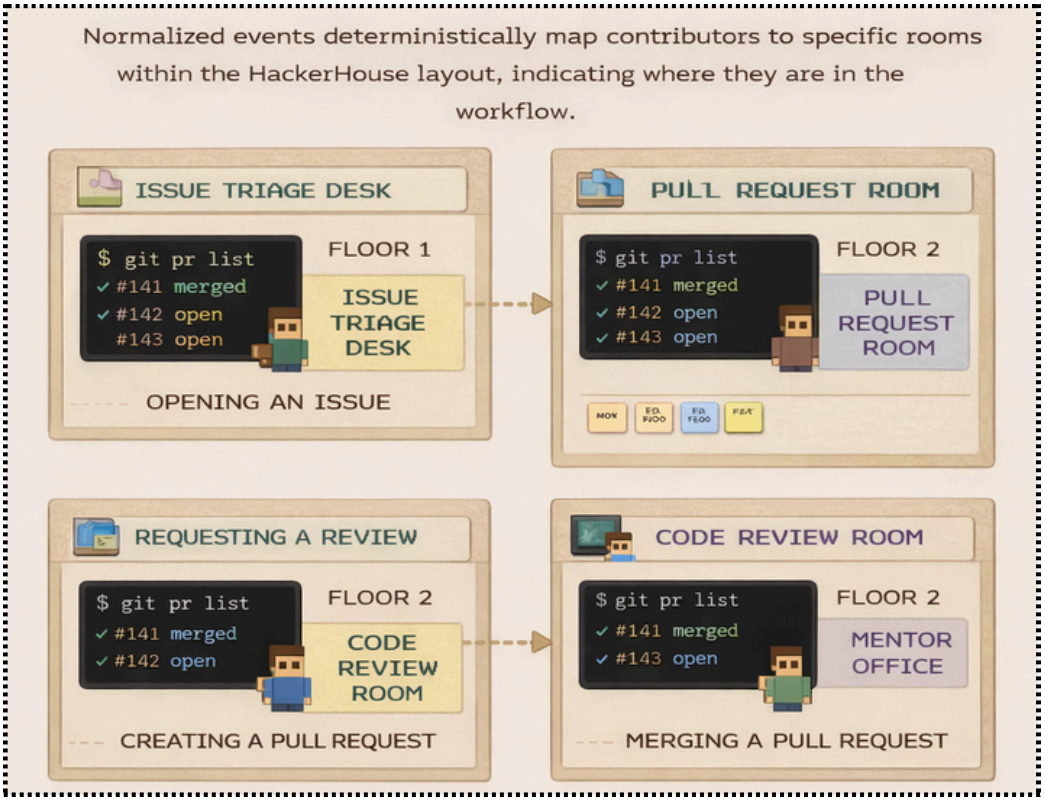
    # 3. Display name similarity (token set ≥ 0.6)
    for c in candidates:
        if any(token_set_ratio(actor_name.lower(), a.lower()) / 100 >= 0.6 for a in c.aliases):
            return c.id, 0.60

    # 4. No match
    return create_unlinked_identity(actor_id, source, db), 0.0
```

Identity resolution follows a staged approach: **exact platform match, fuzzy username matching, and token based display name similarity.**
Each stage assigns a confidence score and falls back gracefully, with unresolved identities explicitly surfaced.

6. Room Routing and Avatar State

Identity resolution follows a staged approach: **exact platform match, fuzzy username matching, and token based display name similarity.**
Each stage assigns a confidence score and falls back gracefully, with unresolved identities explicitly surfaced.




```
ROOM_MAP = {
    "issue_opened":         {"floor": 1, "room": "issue_triage_desk"},
    "bug_bounty_submitted":{"floor": 1, "room": "bug_bounty_terminal"},
    "ci_run":               {"floor":  1,  "room":  "cicd_server_room"},
    "pr_opened":            {"floor":  2,  "room":  "pull_request_room"},
    "review_requested":     {"floor":  3,  "room":  "code_review_room"},
    "pr_merged":            {"floor":   3,   "room":   "mentor_office"},
    "mentor_assigned":      {"floor":   3,   "room":   "mentor_office"},
    "mentor_handoff":       {"floor": 3, "room": "strategy_war_room"},
                                        }
}

defupdate_avatar_state(event: Event, contributor_id: str, db):
    room=ROOM_MAP.get(event.event_type, {"floor": 1, "room": "activity_monitor"})
    db.execute("""
        INSERTORREPLACEINTO avatar_state
        (contributor_id,floor, room, last_event, updated_at)
        VALUES(?,?,?,?,?)
    """,(contributor_id,room["floor"], room["room"], event.event_type, datetime.utcnow()))
    db.commit()
```

7. WebSocket Message Contract

All frontend/backend communication uses the following typed message format. This contract is locked before Week 6 so frontend integration has no discovery phase.

Snapshot (sent on every new connection):

```
{
  "type": "snapshot",
  "data": {
    "contributors": [
      {
        "id": "contrib_123",
        "display_name": "Rudra Pratap Singh",
        "floor": 2,
        "room": "pull_request_room",
        "identity_linked": true,
        "last_event": "pr_opened",
        "score": 42
      }
    ],
    "room_occupancy": {
      "pull_request_room":  3,
      "code_review_room":   1,
      "issue_triage_desk": 5
    },
    "generated_at": "2026-03-16T10:22:00Z"
  }
}
```

Avatar move update (live event):

```
{
"type": "avatar_move",
"contributor_id": "contrib_123",
"floor": 3,
"room": "code_review_room",
"event_type": "review_requested",
"timestamp": "2026-03-16T10:25:00Z"
}
```

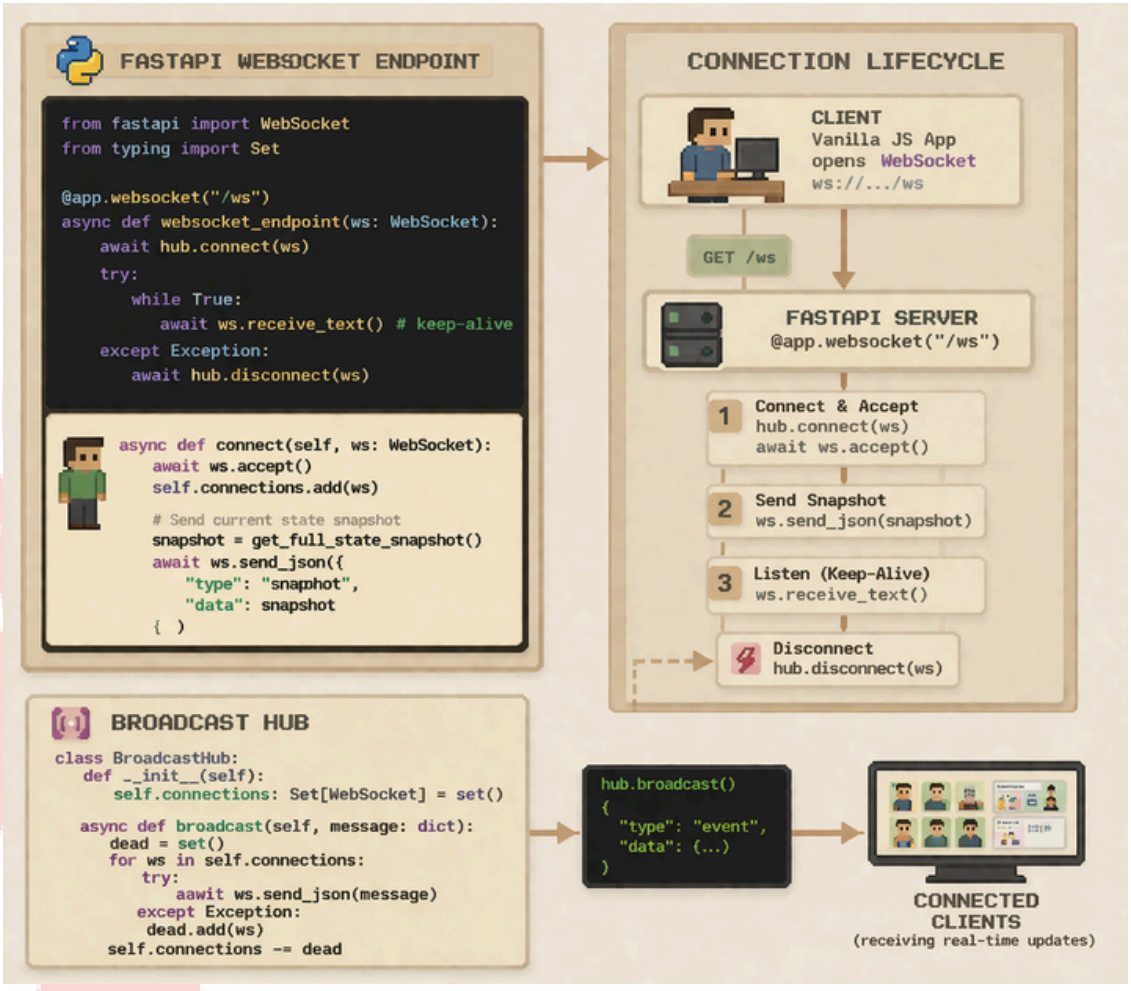
Identity flag (maintainer action required):

```
{
  "type": "identity_flag",
  "suggested_link": {
    "source_a": {"platform": "github", "username": "rudra-rps"},
    "source_b": {"platform": "slack", "user_id": "U0123ABC"},
    "confidence": 0.75
  }
}
```

Leaderboard update:

```
{
  "type": "leaderboard_update",
  "contributor_id": "contrib_123",
  "score": 47,
  "delta": 5,
  "last_event": "pr_merged"
}
```

8. WebSocket Broadcast Hub



9. BLT Signal Adapter (Polling)

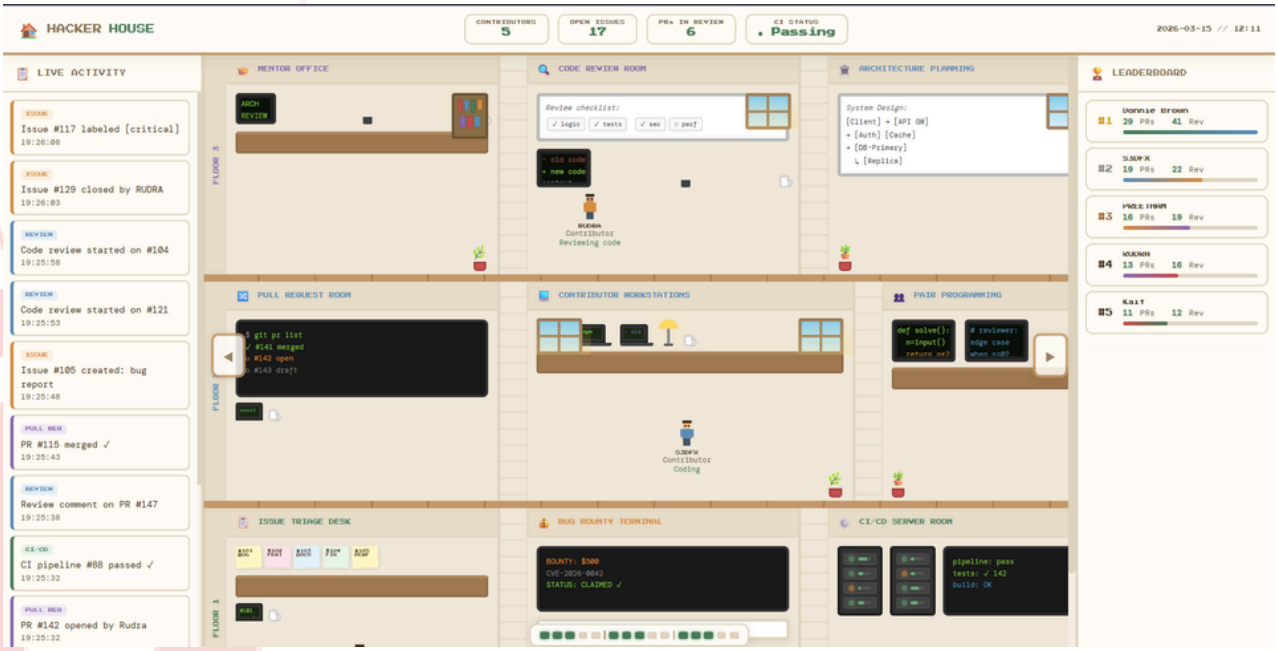
Unlike GitHub and Slack, BLT exposes a REST API rather than webhook based event delivery. The system integrates BLT signals by periodically polling endpoints such as /bugs and /hunts, which provide structured data on bug reports, contributor activity, and ongoing hunts.

A last_seen_id mechanism ensures that only newly created or updated records are processed, preventing duplication across polling cycles. Retrieved records are transformed into the normalized Event schema, allowing BLT activity to be treated identically to GitHub and Slack events within the pipeline. This adapter is isolated from the core ingestion layer and is explicitly additive temporary BLT API failures do not impact realtime processing of GitHub or Slack events.

10. Frontend Visualization Engine

The three floor layout is rendered using SVG/Canvas with state driven avatar positioning. Avatar movement is committed only after a 5 second debounce window to prevent UI thrashing under burst activity.

INITIAL UI IDEA

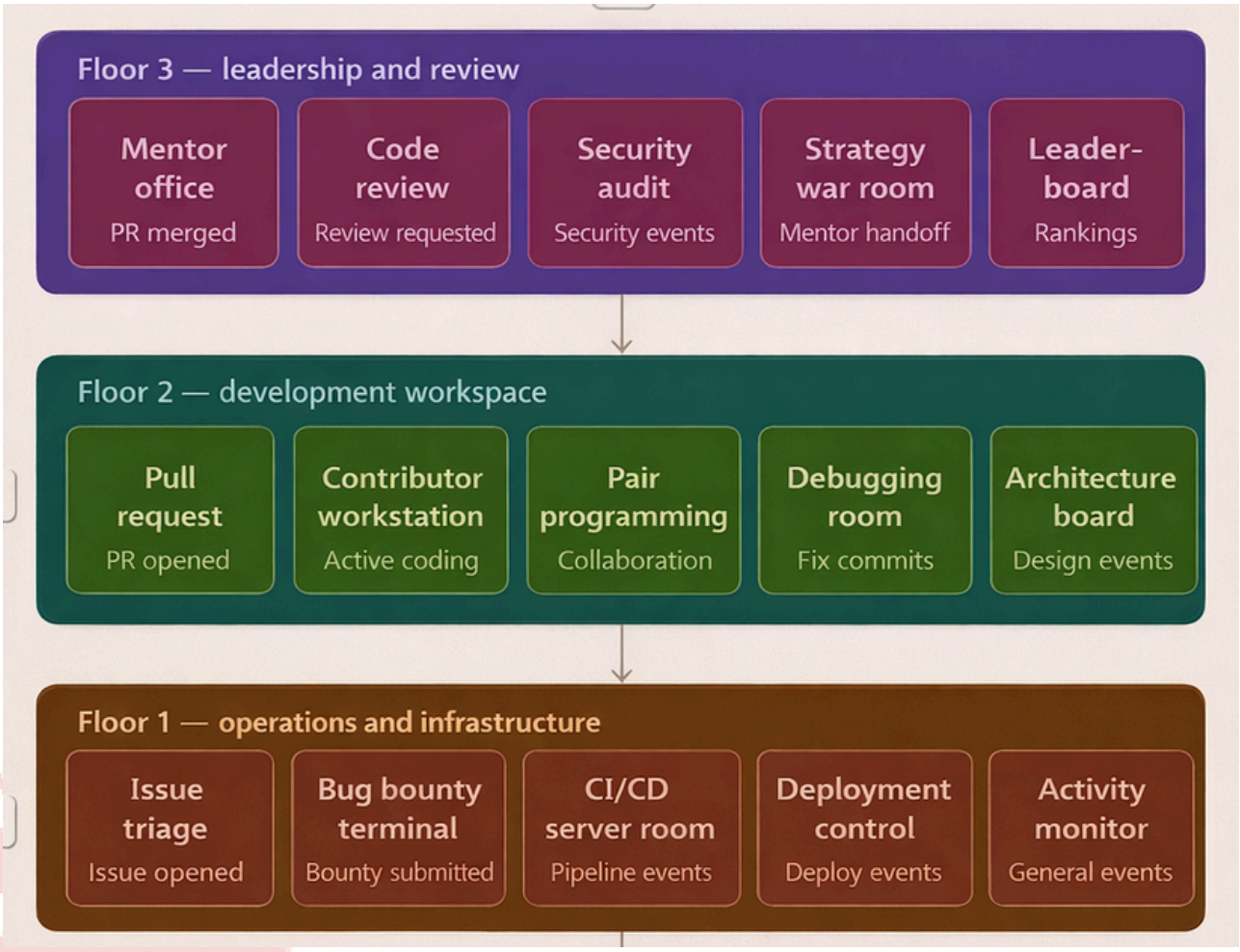


HOUSE LAYOUT

Floor 3: Leadership and Review
Mentor Office · Code Review Room · Security Audit Room · Strategy War Room · Leaderboard Wall

Floor 2: Development Workspace
Pull Request Room · Contributor Workstations · Pair Programming Area · Debugging Room · Architecture Board

Floor 1: Operations and Infrastructure
Issue Triage Desk · Bug Bounty Terminal · CI/CD Server Room · Deployment Control Center · Activity Monitor



Room occupancy is visually capped at 12 avatars per room. Beyond that, a +N counter is displayed. Only changed rooms are redrawn on each update cycle not the full canvas.

Event	Room
Issue opened	Floor 1: Issue Triage Desk
Bug bounty submitted	Floor 1: Bug Bounty Terminal
CI/CD event	Floor 1: CI/CD Server Room
Deploy event	Floor 1: Deployment Control Center
PR opened	Floor 2: Pull Request Room
Review requested	Floor 3: Code Review Room
PR merged	Floor 3: Mentor Office
Mentor assigned	Floor 3: Mentor Office
Mentor handoff/pause	Floor 3: Strategy War Room
Unknown event	Floor 1: Activity Monitor

Deployment Architecture

The system is deployed using **Docker Compose**, enabling a single command local setup.

This includes:

- FastAPI backend service
- SQLite volume for persistent local storage
- Static frontend served alongside the backend

SQLite is used as the default persistence layer to maintain a zero-configuration, local first development experience. This aligns with the expected scale of BLT contributors (tens to low hundreds) and avoids unnecessary operational overhead.

Frontend Resilience

Scenario	Behavior
WebSocket disconnect	Reconnects after 2s; fresh snapshot sent automatically on reconnect
Out of order events	Flagged with stale indicators in activity panel
WebSocket unavailable beyond threshold	Client falls back to periodic HTTP polling
Unresolved contributor identity	Avatar visible with indicator; maintainer can confirm or reject from overlay

Testing Strategy

unit test (pytest)

Each event adapter tested against real fixture payloads captured from GitHub and Slack webhook deliveries.
Identity resolver tested with exact match, Levenshtein match, Jaccard match, and no match cases using the 30 real contributor pairs mapped during pre proposal investigation.
Room routing tested for all 10 event_type mappings, including the unknown fallback.
Leaderboard projection tested for correct accumulation, zero point events, and conflict resolution.

Integration Tests

Full chain: webhook delivery → HMAC verification → normalization → SQLite persistence → WebSocket broadcast.
Snapshot delivery on fresh client connection. Snapshot recovery after WebSocket disconnect and reconnect. Idempotency: duplicate webhook delivery produces no duplicate event in SQLite.

Load Tests (locust)

200 concurrent WebSocket clients sustained for 10 minutes. Sustained 50 events/min ingest rate. Target: p99 WebSocket broadcast latency < 200ms. Results logged to load_test_results/ and included in final deliverables.

Coverage target: 80% on ingestion and normalization modules.



Risk and Mitigation

Risk	Severity	Mitigation
Slack API rate limits	High	Exponential backoff: 1s → 2s → 4s → 8s → max 32s with ±500ms jitter. Dead letter queue capped at 500 events before drop with structured log entry.
Missed GitHub events during downtime	Medium	Track last_seen_event_id per repository in SQLite. Periodic 90-second polling of /events API detects gaps.
Contributor identity fragmentation	Medium	Three stage resolution pipeline (email → Levenshtein → Jaccard) with rapidfuzz. Unresolved contributors remain visible with explicit indicator. Maintainer confirmation flow surfaced in overlay.
BLT API instability	Medium	Polling adapter is isolated. BLT signals are explicitly additive their absence does not break GitHub/Slack pipeline. Polling errors are logged and skipped, not fatal.
Canvas performance under high occupancy	Low	Per room avatar cap of 12 with +N counter. Dirty region redraw only changed rooms are updated, not the full canvas.
COSMIC components with breaking alpha changes	Low	All cosmic integrations (if added post GSoC) are isolated in adapters. Core pipeline is not COSMIC dependent.

Measuring Success

Metric	Target	How Collected
Time to ecosystem understanding	Maintainer understands current state in < 5 seconds	Timed observation during demo with BLT maintainers
Event delivery latency	WebSocket update within 200ms of event processing	Measured during load testing with ~200 concurrent clients
Identity resolution	80%+ of contributors correctly linked across platforms	Ratio of unified to unlinked avatars in 30 pair test dataset
Maintainer tool switch reduction	Measurable drop in platform context-switching	Pre vs. post adoption maintainer feedback survey
End to end pipeline reliability	99%+ of webhook events ingested and persisted without loss	Integration test results across GitHub and Slack event types

Deliverables

Each deliverable is demonstrable and independently testable:

D1: Working Python event pipeline Webhook events received, verified, normalized, and persisted within 500ms. Demonstrated with live GitHub webhook against local server.

D2: Slack integration Slack mentions tied to issues and PRs are ingested, normalized, and identity mapped.

D3: Live streaming broadcast Connected clients receive WebSocket avatar updates within 200ms of event processing. Demonstrated with three concurrent clients.

D4: HackerHouse canvas renderer Three-floor layout with live room occupancy, debounced avatar movement, and +N overflow counters.

D5: End to end demo

1. Open a GitHub issue → avatar appears in Floor 1 (Issue Triage Desk)
2. Open a PR from that issue → avatar moves to Floor 2 (Pull Request Room)
3. Request review → avatar moves to Floor 3 (Code Review Room)
4. Merge PR → avatar moves to Floor 3 (Mentor Office)

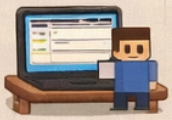
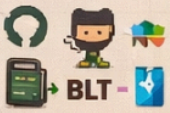
D6: Leaderboard and metrics panel

Live contributor score, ecosystem status indicators, and room occupancy heatmap.

D7: Deployment ready Docker Compose

Single command launch of full stack: FastAPI, SQLite volume, and vanilla JS frontend. Includes contributor documentation for adding new event adapters.

PROJECT TIMELINE

Pre-GSOC & Community Bonding	Week 1 May 25 – May 31	Week 2 June 1 – June 7	Weeks 4–5 June 15- – June 21	Weeks 6–12 Midterm Evaluation
Pre-GSOC & Community Bonding	**BLT + WebSocket Prototype**	**GitHub Ingestion Start**	**Slack + GitHub MVP**	**Leaderord Pipeline**
Setup Backend Infrastructure	BLT API Integration	Add HMAC Verification	Normalize Slack Events	Validate Real Data Flow
Identity Resolution Pipeline	Validate WebSocket Contract	Deduplicate First 3 GitHub Events	Ingest GitHub PR Events	Connect WebSocket
Snapshot Endpoint				
Initial Fixture Preparation				

BLT + WebSocket Prototype → **GitHub Ingestion Start** → **Normalize SQLite Event Storage** → **Slack + GitHub MVP** → **LeaderBoard Pipeline**

- BLT API Integration
- Snapshot Endpoint
- Initial Fixture Preparation
- Add HMAC Verification
- Deduplicate First 3 GitHub Events
- Normalize Slack Events
- Ingest GitHub PR Events

BLT + WebSocket Prototype → **GitHub Ingestion Start** → **Slack + GitHub MVP** → **Frontend Integration**

- BLT + WebSocket Prototype
- GitHub Ingestion Start
- GitHub Ingestion Start
- Slack + GitHub MVP
- Normalize SQLite Event Storage
- and..

Project Timeline

> **Note on schedule:** End-semester exams run from late May to mid-June. Weeks 1–3 are intentionally lighter. Full pace begins from Week 4.

Learning Phase (Pre-GSoC + Community Bonding)

Before Week 1 begins, I will:

- Complete a working prototype of the identity resolution pipeline against real BLT contributor data.
- Confirm BLT REST API endpoints and validate the polling adapter.
- Freeze the WebSocket message contract (as documented above) with mentor sign off.

Week 1 (May 25 – May 31, ~15 hours)

- Finalize event to room mapping with mentor feedback.
- Freeze MVP scope and confirm non goals list.
- Set up repository structure, CI pipeline, and baseline FastAPI skeleton.

Week 2 (June 1 – June 7, ~15 hours)

- Define and validate normalized event schema and Pydantic models.
- Implement SQLite schema (all four tables) with migration scripts.
- Prepare real fixture payloads for GitHub, Slack, and BLT adapter inputs.

Week 3 (June 8 – June 14, ~15 hours)

- Implement GitHub webhook ingestion with HMAC-SHA256 verification.
- Add deduplication with idempotency keys.
- **Milestone:** First 3 GitHub event types (issue_opened, pr_opened, pr_merged) ingested and persisted.

Week 4 (June 15 – June 21, ~40 hours)

- Integrate Slack Events API verification and ingestion. Add BLT 60 second polling adapter.
- Implement identity resolution pipeline (all three stages) with rapidfuzz.

Week 5 (June 22 – June 28, ~40 hours)

- Build async processing queue and state reducer. Implement
- FastAPI WebSocket broadcast hub and connection registry.
- Add snapshot on connect endpoint for frontend recovery.

Milestone: Backend pushes live room state updates end to end.

Week 6 (June 29 – July 5, ~40 hours)

- **Wire frontend to WebSocket and validate message contract against real backend.**
- Fix any contract gaps discovered during integration .
- Add debouncing/throttling for burst event windows.
- Add stale data indicators and fallback polling path.

Midterm Evaluation Deliverables

- Working GitHub webhook ingestion with HMAC verification.
- Slack event ingestion and normalization baseline.
- SQLite-backed storage for normalized events, identities, and room state.
- FastAPI WebSocket broadcast plus snapshot endpoint.
- End to end backend flow showing room state updates from live events.
- Frontend wired to confirmed WebSocket contract.

Week 7 (July 6 – July 12, ~40 hours)

- Complete room occupancy rendering and avatar positioning (contract is confirmed, not discovered).
- Add activity feed payloads.
- Add leaderboard projection API.

Week 8 (July 13 – July 19, ~40 hours)

- Implement full leaderboard pipeline (GitHub + BLT signals).
- Add mentor lifecycle projection (assigned, handoff/pause, rematch).
- **Milestone:** Mentor and leaderboard signals appear consistently in UI.

Week 9 (July 20 – July 26, ~40 hours)

- Add reliability layer: exponential backoff retries, dead letter handling.
- Improve event ordering and idempotent replay protection.
- Add health endpoints and structured logging for all ingestion paths.

Week 10 (July 27 – August 2, ~40 hours)

- Write unit tests for normalizers, routing, and identity mapping.
- Add integration tests for webhook to state and WebSocket delivery flow.
- Run locust load tests targeting ~200 WebSocket clients.

Week 11 (August 3 – August 9, ~40 hours)

- Address test failures, optimize hot paths, tighten error handling.
- Complete Docker Compose setup and reproducible local demo scripts.
- Prepare contributor documentation for adding new event adapters.

Week 12 (August 10 – August 16, ~30 hours)

- Final polish, mentor review fixes, and documentation cleanup.
- Prepare final demo walkthrough (full D5 scenario with real events).
- Publish final report with delivered MVP, confirmed limitations, and post GSoC roadmap.

Buffer and Handoff (August 17 – August 18, ~12 hours)

Handle final feedback and deferred bug fixes. Finalize post GSoC maintenance plan and handoff notes.

Scalability

For the expected BLT scale of 50–200 active contributors, a single Python FastAPI process with SQLite handles early stage load comfortably. For higher traffic, an optional Cloudflare Worker can front the WebSocket hub for edge caching and geographic fan out without changes to the core ingestion service. No architectural changes are required to enable this it is a deployment layer addition.

CONTACT INFORMATION

NAME : Rudra Pratap Singh
EMAIL : 2005ruudrapratapsingh@gmail.com
GITHUB : Rudra-rps
LINKEDIN : Rudra
LOCATION : Gurugram, Haryana, India
TIMEZONE : IST(UTC + 5:30)
Slack : Rudra Pratap Singh
dev.to : rp_dex

STUDENT AFFILIATION

UNIVERSITY : Vivekananda Institute of Professional Studies
DEGREE : B.Tech (IIOT)

GRADUATION YEAR : 2028

BRIEF BIO

I'm a backend developer who genuinely loves distributed systems not just building them, but understanding how they hold up when things get messy.

My work lives mostly in **Go** and **Python**: event driven architectures, data pipelines, API integrations, and lately, LLM tooling and multi-agent systems. I contribute to OWASP-BLT, working on security tooling and real time data systems, which has sharpened both my technical instincts and my appreciation for what makes open source actually maintainable.

A few projects I'm proud of: **Finopoly** (financial education for teens, because money shouldn't be boring), **Trenches** (an AI-agent-driven social network simulator with distributed event sourcing), and **GhostLAN** (a decentralized, offline first LAN gaming platform — anti-cheat and all). Each one started as a genuine problem I wanted to solve.

I also lead the **Eonics Club** at **VIPS-TC**, where mentoring peers has quietly become one of my favourite parts of the week.

This project sits right at the intersection of things I actually care about secure systems, large scale architecture, and developer focused tooling. The skills match, but more importantly, so does the interest.

Why I Am Suited for This Project

Every core layer of HackerHouse maps to something I have already built or debugged: **Event normalization:** BLT-Leaf PR #42 required modeling contribution lifecycle states and surface them to maintainers in a normalized format. That is exactly what the Event schema and adapter layer does in HackerHouse.

The workspace projection in the HackerHouse panel uses the same primitives.

Multi platform integration: I have combined GitHub and Slack event streams in prior tooling work and understand the practical differences in payload structure, rate limiting, and signing verification between the two.

Identity resolution: I manually mapped 30 real BLT contributor pairs across GitHub and Slack before writing a single line of the identity mapper. The three-stage pipeline (email → Levenshtein → Jaccard) comes from that empirical analysis, not from assumption.

Open source workflow understanding: My BLT contributions have given me direct exposure to contributor journeys, maintainer review bottlenecks, and the visibility gaps this project addresses. I am not theorizing about the pain I have experienced it while contributing.

Previous Contributions

OWASP BLT

Pull Requests	Status	Review Status	Notes
Fixed minor inconsistencies in Light mode #10	PR Merged	**Done**	Done
correct tablet breakpoint so header collapses properly#73	PR Merged	**Done**	corrects header and responsiveness
Adding myself as mentee#29	PR Merged	**Done**	
Remove conflicting static.yml Pages workflow causing 404#61	PR Merged	**Done**	Removed conflicting static.yml Pages workflow causing 404 error.
Structured build workflow along with initial UI system improvements for BLT.#59	PR Closed	**Done**	Introduced UI enhancements
feat: mobile-first responsive refactor with animated hamburger drawer	PR Closed	**Done**	Through this learnt about how to make UI more descriptive and efficient as this PR was closed and the other PR provided better solution.
feat: Add 'How It Works' explainability page with 8 sections#138	PR Merged	**Done**	added “How it Works” section
feat: Add comprehensive PR readiness analysis system- Add timeline tr...#42	PR Merged	**Done**	Introduced PR readiness tool for improved workflows
Flakiness detection init#281	PR Open	**Ready for Review**	Under Review introduces flakiness detection
Fix theme toggle issue on "How it works " page and Implement Cache Busting#259	PR Open	**Ready for Review**	Under Review as of now taught me about cache busting for optimal approach.
feat: mobile-first responsive refactor with animated hamburger drawer#23	PR Merged	**Done**	Fixed Browse Now as proper anchor linking to https://github.owaspblt.org/BLT-Ideas/
added GET /api/assignments endpoint#100	PR Open	**Ready for Review**	• Added GET /api/assignments endpoint • Returns active mentor to contributor mappings from mentor_assignments table

Responsible Use of AI Tools

AI tools were used selectively to improve clarity, structure, and presentation of this proposal. They were primarily used for refining language, organizing sections, and ensuring that explanations are concise and readable for reviewers.

All technical content including system architecture, event pipeline design, identity resolution strategy, data modeling, and implementation planning originates from my own hands on work and investigation within the OWASP BLT ecosystem. The design decisions reflect practical experimentation, prior contributions, and direct understanding of the challenges faced by contributors and maintainers.

Where AI assistance was used, the output was carefully reviewed, validated, and modified to ensure technical correctness, feasibility, and alignment with real system constraints. No architectural decisions, algorithms, or implementation approaches have been generated blindly or adopted without verification.
The intent behind using AI tools was not to replace original thinking, but to improve communication quality and ensure that the proposal accurately conveys the work I am capable of implementing during the GSoC period.